PROJECT PROPOSAL & REPORT

Math Jigsaw

Making Mathematics Amusing for Younger Students

A low-fidelity native application proposal leveraging modern UI components and gamified visual puzzle logic to combat early childhood mathematics anxiety and engage device-centric learners.

Table of Contents

I. About the project

1. Why?

Many studies ([1], [2], [3], [4]) demonstrate that over the years, more and more youth see math as a chore or a source of anxiety.

We want to change that, bringing back the love of numbers and the joy of solving problems to younger students —in the most efficient way possible, of course! 😉

2. How?

Studies highlight a shifting educational landscape:

Note: While these global studies highlight widespread trends, observation suggests these behavior patterns heavily translate to the Philippine context as well.

1. Adolescents spend an average of 5.6 hours per day on their smartphones [5].
2. Physical textbook usage is decreasing rapidly among modern students [6].
3. Consequently, a large majority of educators bypass assigned textbooks [7] to source or develop alternative content.

Conclusion: Traditional teaching approaches are losing the battle for youth attention.

What can we do about it?

The Status Quo

Keep complaining about their screen time.

The Opportunity

Meet them where they already are: through technology.

Let's change the game. Literally.

We are building a **mobile application** designed to turn math into an addictive, puzzle-based game.

Why a mobile app instead of a desktop platform? Because modern youth carry their phones everywhere. The goal is to occupy the maximum amount of devices possible.

> **Our Philosophy:** Every single second a student spends engaging with our app instead of consuming mindless content is a massive victory for the education system.

3. Constraints

Like any projct, we have a few constraints to consider:

1. Budget : We have very limited budget
2. Time : We have a very tight timeline (3 weeks) to develop a working prototype
3. Resources : We have a small team of 2 developers and 1 designer

We need to make sure that we can deliver a working prototype within the given constraints.

II. The Game

"A goal without a plan is just a wish."

— Antoine de Saint-Exupéry

1. Game Concept

The first iteration of the game as we envision it , is as follows :

At the beginning , the player is presented with an empty puzzle board, and a quick mental math problem to solve. Once solved properly, the puzzle piece is placed on the board , and the player is presented with another problem. The game continues until the puzzle is completed.

Why Jigsaw ?

We evaluated the different types of puzzles that could be used in the game, and we decided to go with a **jigsaw puzzle**. This is because jigsaw puzzles are easy to understand and can be completed in a short amount of time, which is ideal for simple mobile sessions.

The point is to make the game as **simple , intuitive and pleasing** as possible, so that the players never get bored or frustrated. The game should be a fun and engaging way to practice math skills, and we believe that a jigsaw puzzle is the perfect way to achieve this.

In that sense, as we'll see in the next parts, we spend a lot of time thinking about user interface and experience, as well as the different mechanics and effects we can use to make the game more engaging and fun.

Because the real **challenge** here, is that we have to get students to play the game - and keep them playing -, which means literally means **competing with the other apps on their phones**, which are sadly way more stimulating and addictive, such as TikTok, Instagram, etc... *Which of course aren't as good for their brains as our math game*

2. Sketch (Hand-Drawn)

We then drew some sketches of how the features we describe previously could be presented in an application

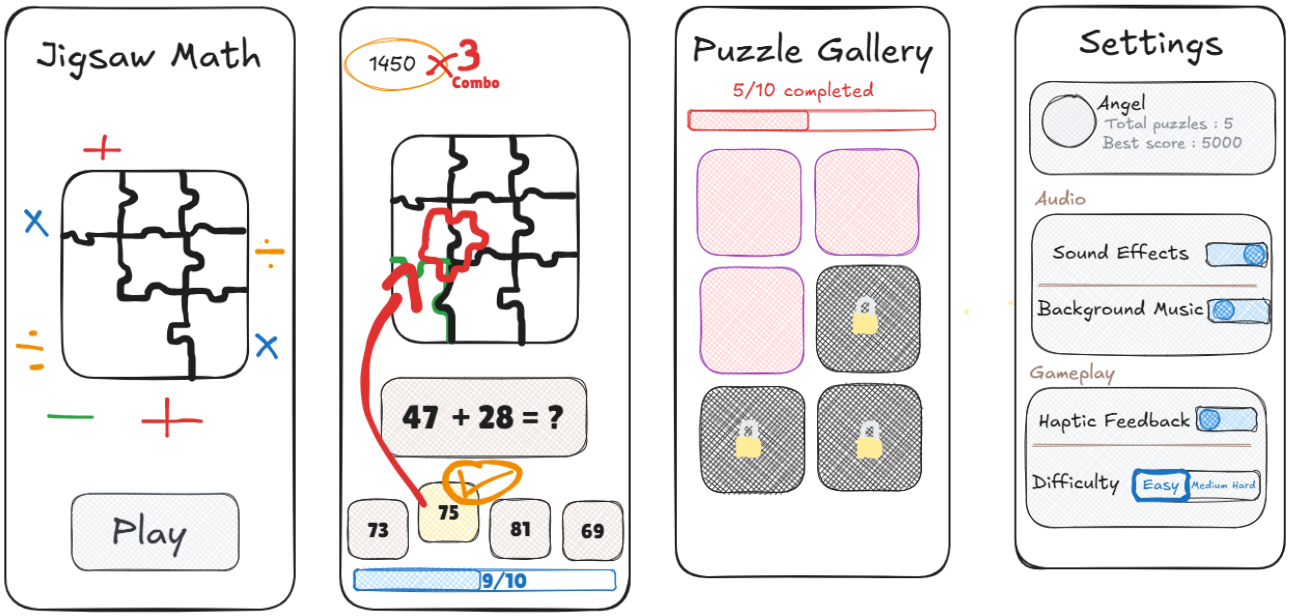

Figure 1: A side-by-side look at the different in-game screens.

3. Mockups (AI-Generated)

Here's how the user interface could look like, that the AI generated a visual for based on our hand sketches. The AI-generated mockups are not final, but they give us a good idea of how the app could look like.

Indeed, it's important to have an idea of the final product before starting the work any project.

They key takeaway from these mockups, is how colors, shapes and simple effects can turn a sketch into an attractive game interface, and so that after making the core of the game, we will have to do the same too !

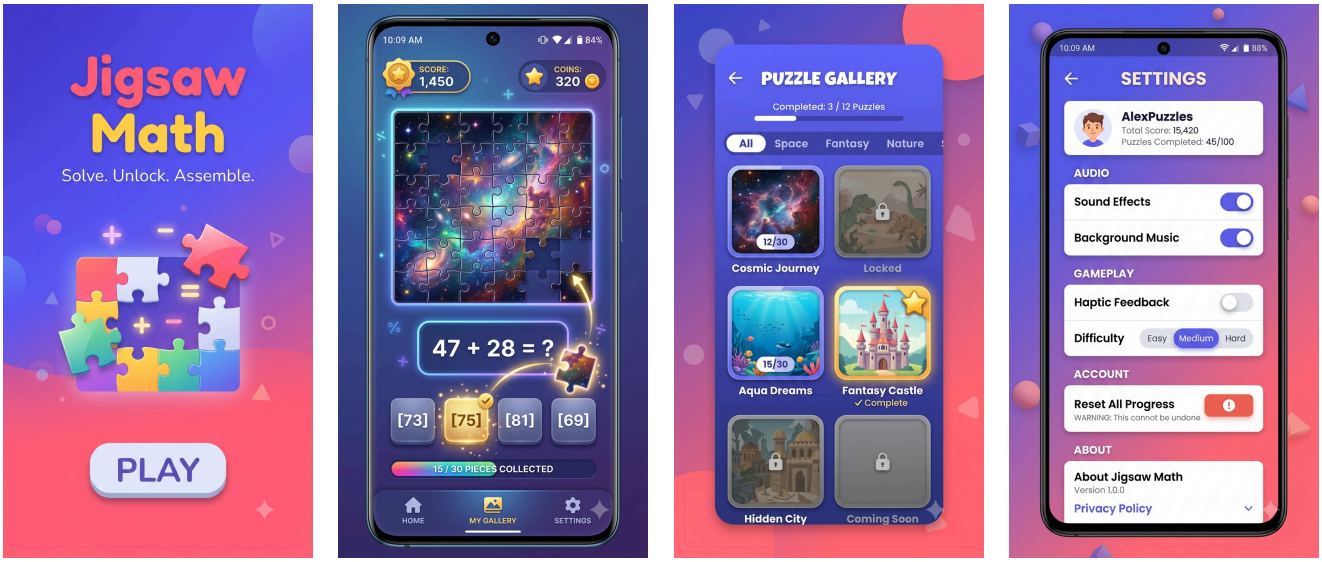

Figure 2: A side-by-side look at the AI-Generated user interface.

III. Technical Details

1. Technology Stack

Now that we have a clear understanding of the problem, let's discuss the technical details of our solution.

We want to make a mobile application, however there's currently 2 types of phones on the market - Android (70% market share) and iOS (30% market share). iOs is what powers iPhones, while Android powers a wide variety of devices from different manufacturers - Samsung, Huawei, Xiaomi, Oppo, etc.

Figure 3: The eternal debate - Apple vs Android

To deliver a working prototype within our tight constraints, we evaluated three potential development paths based on implementation speed, learning curve, and final output quality:

Approach	Pros & Cons	Learning Curve
Native Android (Kotlin + Compose)	Targets android only. Jetpack Compose allows rapid UI development with modern, declarative code.	**Fast** (Excellent documentation and resources available)
Cross-Platform (Flutter / React Native)	Reaches both iOS and Android, but requires learning an entirely new ecosystem and configuration from scratch.	**Steep** (Too risky for a 3-week sprint)
MIT App Inventor	Extremely fast block-based prototyping, but highly restrictive for building a polished, complex game architecture.	**Immediate** (Too limited for scaling)
XCode / SwiftUI	Targets iOS only. SwiftUI allows rapid UI development with modern, declarative code.	**Fast** (Excellent documentation and resources avail-

		able, but requires a Mac to develop and test + fees)

Retained Approach: Native Android (Kotlin & Jetpack Compose)

Given our strict **3-week timeline** and small team, maximizing development speed is paramount. While MIT App Inventor offers rapid block prototyping, it lacks the flexibility needed for a game layout. Cross-platform options present too steep a learning curve for a short sprint. Therefore, we retained **Kotlin with Jetpack Compose**—allowing us to leverage abundant, high-quality educational resources to quickly deploy a responsive native app for the majority market share.

2. Tools

Since we are developing a mobile application, we need to use the following tools:

- Android Studio : The official IDE for Android development. It provides a complete set of tools for building, testing, and debugging Android apps.

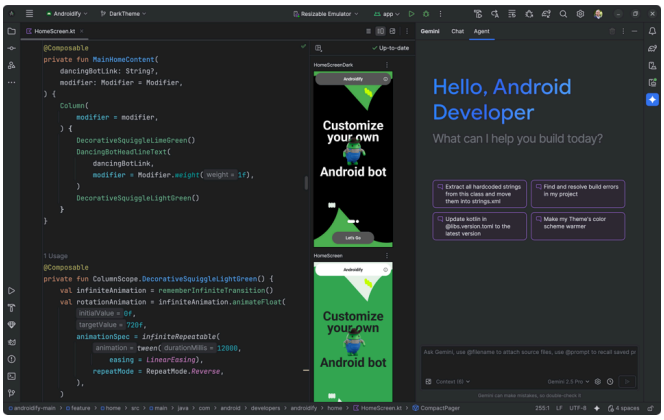

Figure 4: Android Studio - The official IDE for Android development

- GitHub : A web-based platform for version control and collaboration. It allows us to manage our codebase, track changes, and collaborate with team members.

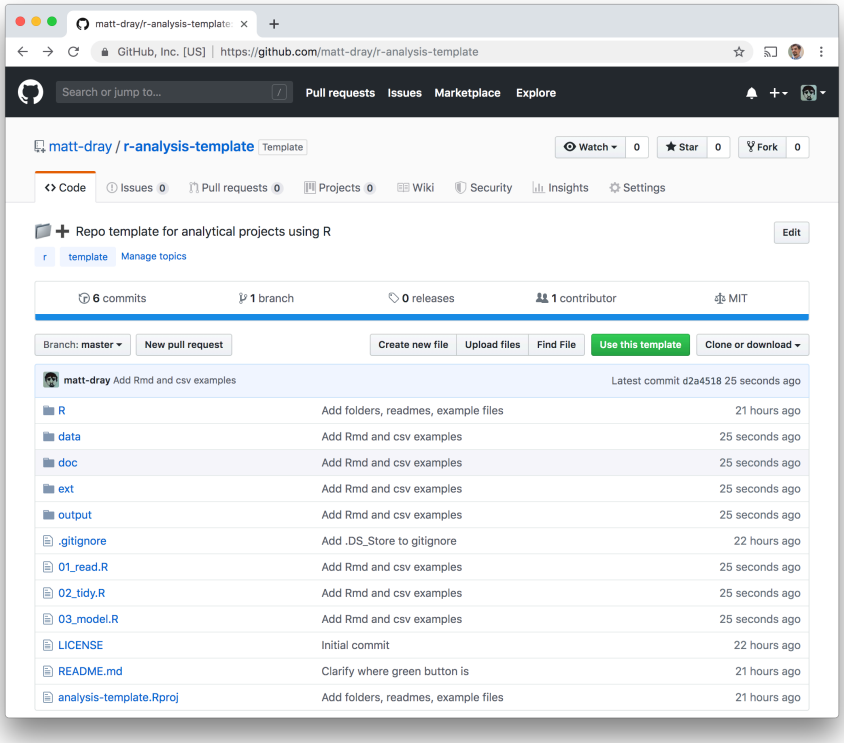

Figure 5: GitHub - A web-based platform for version control and collaboration

- AI : We will leverage AI tools to assist in code generation, debugging, and optimization. This will help us speed up development and ensure high-quality code.

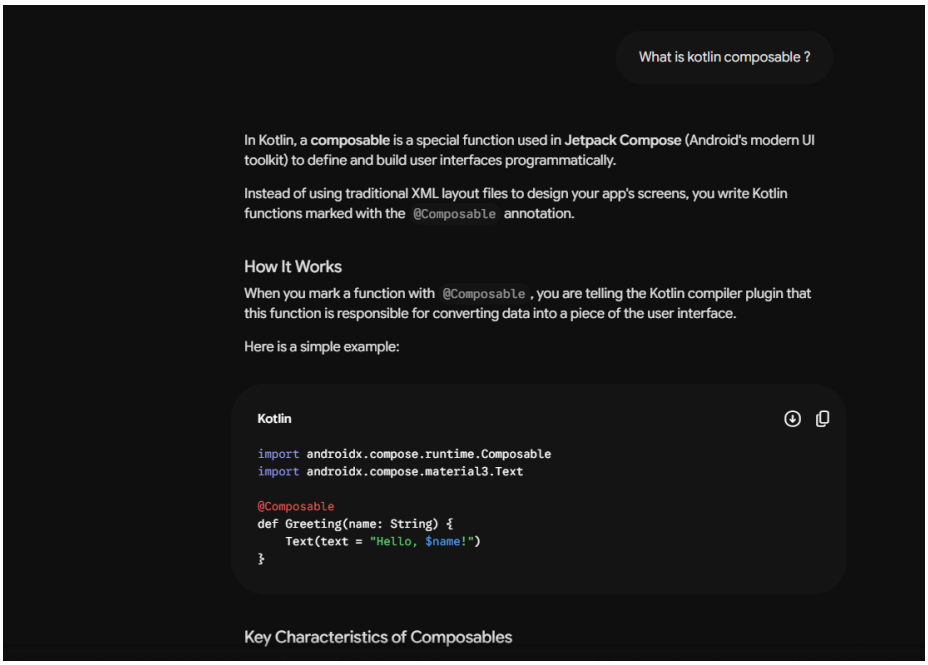

Figure 6: AI - Leveraging AI to learn app development.

3. Developement Environment

There are various operating systems available for different types of computers. The PCs we had access to were running **Windows**..

Microsoft defines Windows as a general-purpose operating system designed to run on personal computers, including desktops, laptops, and tablets. It provides a graphical user interface (GUI), virtual memory management, multitasking capabilities, and support for many peripheral devices.

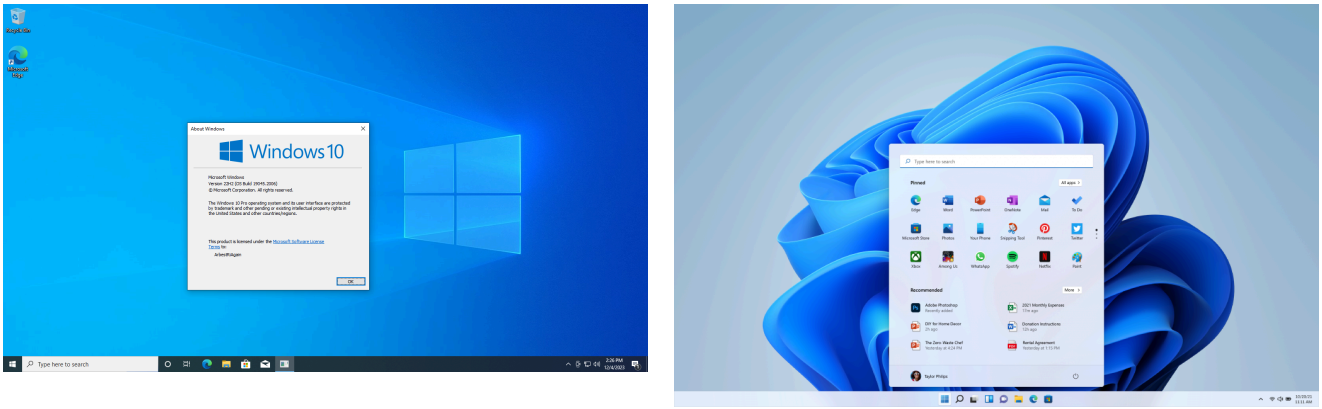

Figure 7: Windows 10 and Windows 11

4. Installing Android Studio

We install Android Studio through the official website https://developer.android.com/studio.

All we have to do is download the installer, and follow the instructions. We helped ourselves with this youtube video https://www.youtube.com/watch?v=XmmMXYBzlns

Figure 8: Android Studio - Splash screen on launch

5. Programming Language

Android studio supports multiple programming languages and frameworks, but we will be using **Kotlin** for our project.

See Section IV for more details

IV. Kotlin & Jetpack Compose

1. History of Android Development

For many years, Android development was primarily done using Java, a widely-used programming language known for its portability and robustness. It powers a wide range of applications, from web servers, mobile apps to games like Minecraft.

However, it presented some challenges for Android developers, and the language was often criticized for encouraging old programming practices that could lead to less efficient and more error-prone code. So in 2017, Google, the company behind Android, announced Kotlin as an officially supported language for Android development, aswell as a new ecosystem that accompanies it called Jetpack Compose.

Basically, developers went from visually designing their apps using a visual editor, and making it interactive with code, to literally writing the entire app in code, including the user interface.

As you can see in the image below, on the left, we can see how developers back in the day had an editor to visually add an place buttons, text fields, images and other elements.

However, on the right, we can see how developers now have to write code to create the same user interface, and then preview it in a separate window.

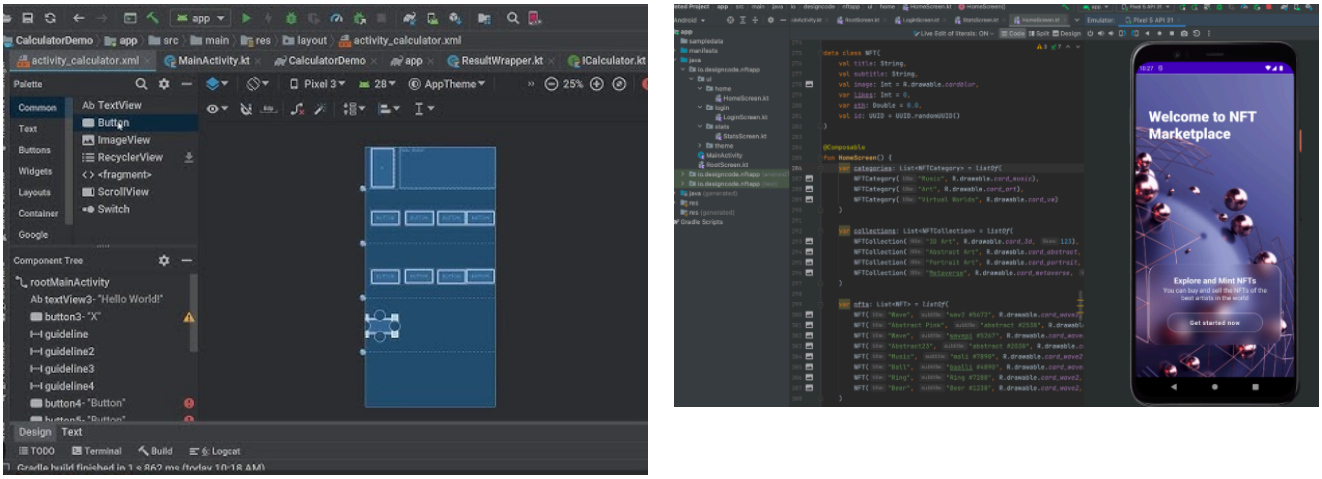

Figure 9: On the left, the old XML-based UI editor. On the right, the new Jetpack Compose preview.

2. About Kotlin

As said before, modern Android development is primarily done by writing code in the Kotlin programming language, so we will be using it for our project too !

3. Using AI to accelerate development

The use of AI in software development has become increasingly prevalent, offering a range of benefits that can significantly enhance the development process.

The latest Android Studio version that we used for our project, includes an AI-powered code assistant that can help developers write code, and even autonomously generate hundred of lines of code based on a human prompt.

Given our tight timeline, and to maximize our productivity, we decided to use it too, since we live in a world where economy rewards the fastest actors, and not specially the most skilled ones…

Below , we present a sample interaction with the AI code assistant, where we asked it to create a visual component, based on text instructions and a screenshot :

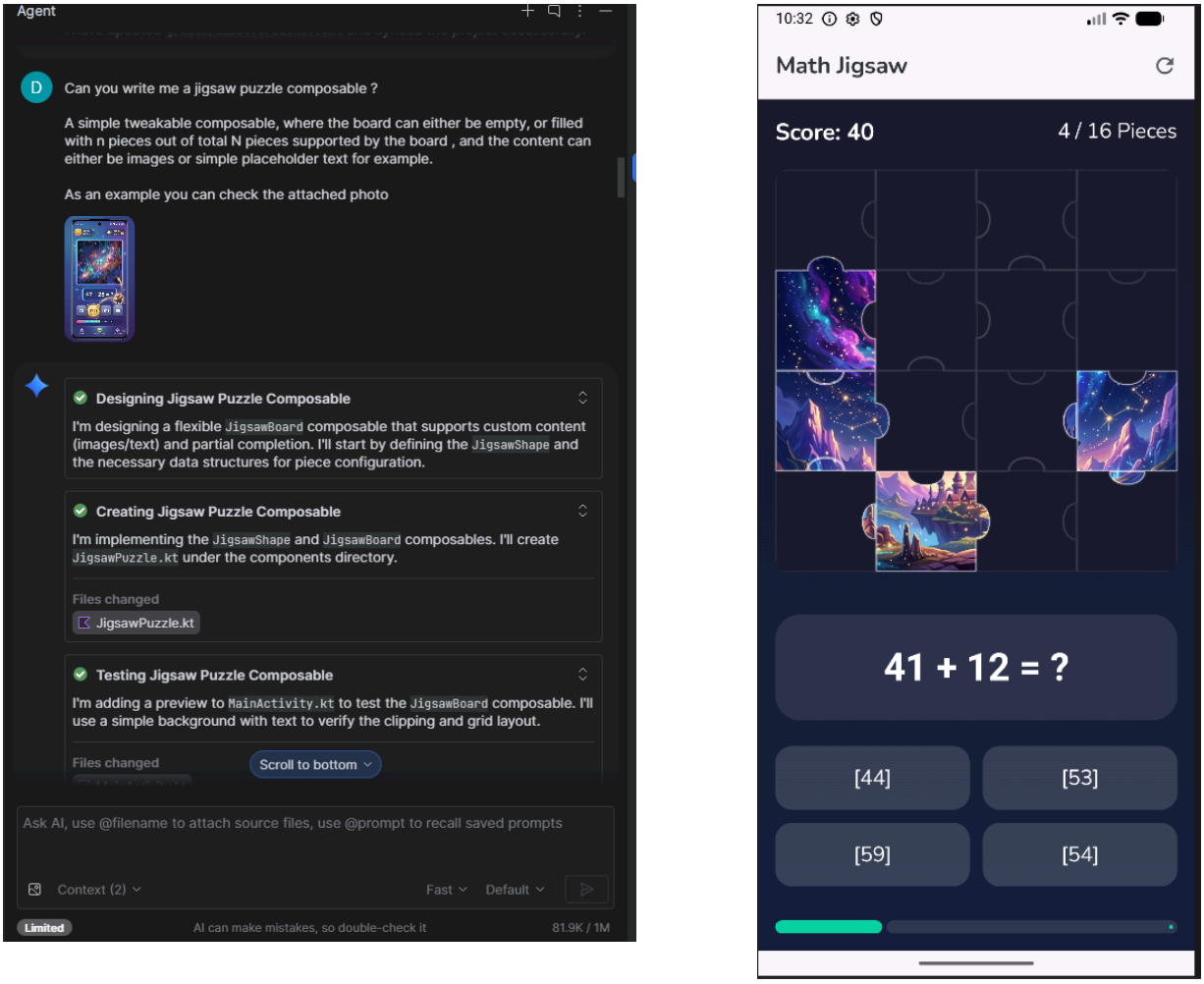

Figure 10: Prompt we gave to the AI code assistant on the Left, and the result it generated on the Right

After a few minutes, and a few prompts to further customize our request, we got the result on the right, which was fully working, had no errors and ran perfectly well.

We can clearly see it's not 1:1 with our mockup, and it is not as polished as we would like it to be, but it is a good starting point. The complicated technical details of the code are handled by the AI, and we can focus on the design and user experience instead !

We automated technical parts of the code, that would otherwise would have taken us hours or days to learn and do manually, which is what companies like Google, Microsoft, and OpenAI are doing to accelerate software development and reduce costs in real life scenarios.

V. Publishing

In order for Android smartphone users to be able to find and download our application, we have to send it to an **Application Store**, which is an **online** service that acts as a sort of catalog for users to safely browse, install, and update apps on their phones. You can think of it like a big city mall where everything is neatly organize and ready for you to get.

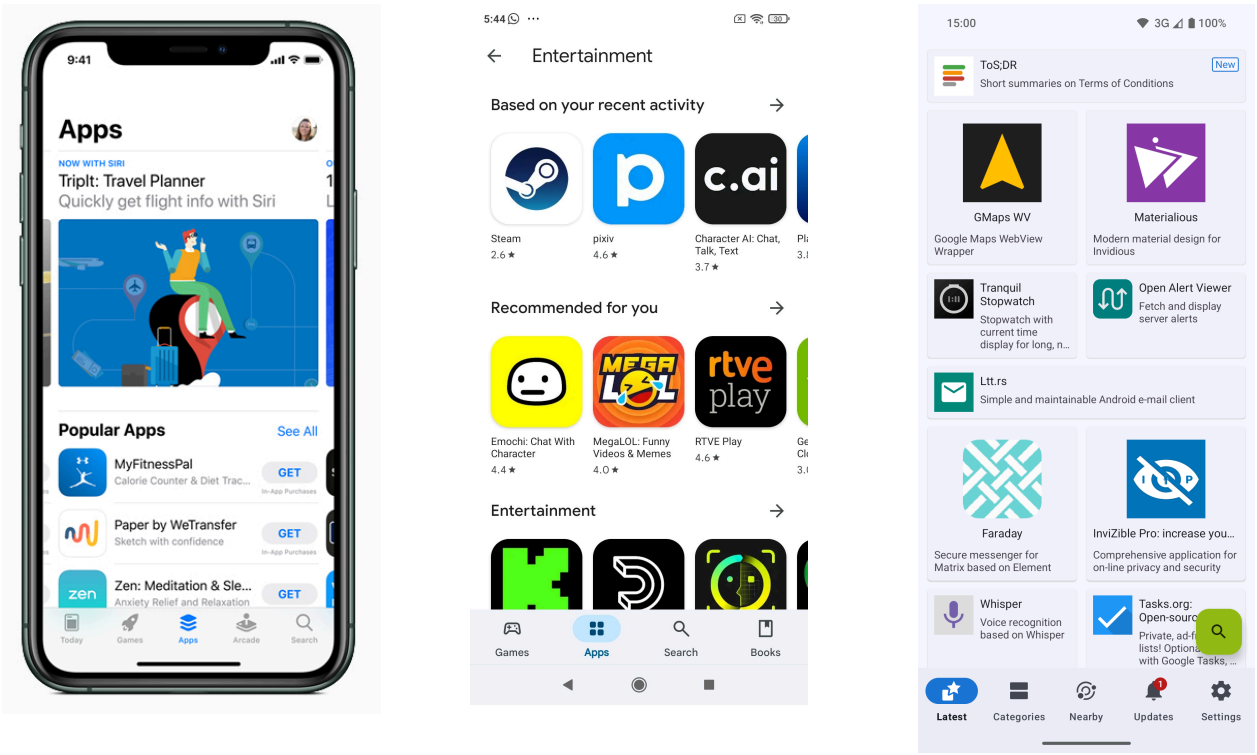

Figure 11: Example of Application Stores : **iOS App Store**, Android **Play Store** and **F-Droid** Store

1. What's "Publishing" ?

In plain terms, **publishing** an app means **making it available to the public**. When developers finish building an app, it exists as a single file on their computer. "Publishing" is the process of uploading that file to a marketplace so that everyday users can find it, click "Install," and have it safely placed onto their devices.

Even if we didn't have enough time to really consider **publishing** the application, we still studied the different possibilities regarding that matter. More precisely, we focused on two marketplaces : "Google Play Store" and "F-Droid". Because they are the simplest , most accessible and widespread.

2. Google Play store

The Google Play Store is the "official" and most popular digital storefront on Android devices. It comes pre-installed on almost every Android phone in the world.

A. Pros

- **Massive Audience:** It has billions of active users. If you want the most people possible to see your app, this is where you go.
- **Ultimate Convenience:** Users don't have to change any settings on their phones to use it. It also handles automatic updates seamlessly.
- **Built-in Security Checking:** Google runs automated scans on apps to catch malware before it reaches a user's phone.

B. Cons

- **Cost and Hurdles:** Google charges a one-time registration fee (25 USD) for developers and takes a percentage cut of any money the app makes.
- **Strict Corporate Rules:** Google has strict policies regarding what an app can do, and they can remove your app from the store if rules change.
- **Privacy Trade-offs:** Google tracks a lot of user data and download habits through the platform.
- **2026 Changes**: Google changed the process, and there's even more complicated steps now. (ID Verification + human testing etc)

3. Option B: F-Droid

F-Droid is an alternative, independent app catalog. It is entirely non-profit and focuses exclusively on **Free and Open-Source Software (FOSS)**.

A. Pros

- **Strict Privacy:** F-Droid does not track what users download, doesn't require a Google account, and bans apps that contain hidden tracking or heavy advertising.
- **Open and Transparent:** Every app on F-Droid must show its "recipe" (its source code) openly. This

B. Cons

- **Harder for Average Users to Install:** Because Google doesn't allow competing app stores inside the Play Store, users have to manually download F-Droid from a website and change a setting on their phone to allow "Unknown Sources."

allows security experts to verify that the app isn't doing anything sneaky behind the scenes.
- **Completely Free:** There are no fees to publish or download apps.
- **Much Smaller Audience:** F-Droid is mostly used by tech-savvy people or privacy enthusiasts. It has thousands of apps compared to Google's millions.
- **Slower Updates:** Because human volunteers manually check and compile the code for safety, app updates can take a few days or weeks to show up compared to the Play Store.

4. Choice

Given the fact, that our app is intended to be educational, we have three possibilities:
- make it free
- make it paid
- make it free but integrate in app-purchases, just like free-to-play videogames.

Currently, the application is more of a proof-of-concept, than really a final product that we could integrate paid features into, so we found out that if we had to publish it in it's current state, it shouldve been into F-Droid, since there's no publishing fees, and we don't need the financial features given by Google Play Store. Addtionally ,on F-Droid, potential users can download it for free and give feedback in order for us to improve the application, as well as contribute to our project.

VI. Difficulties

During the creation of the app, we faced so many difficulties

1. The tools are so heavy

The tool that we used, Android Studio, is so heavy and complex , it requires having a powerful computer, and a very high network bandwidth to work properly.

Since the computer we had access to, wasn't powerful enough, we spent a lot of time waiting, even on the simplest of tasks such as changing small parts of code. It made us waste a lot of time and energy, and the developement would probably have not been possible without the AI developement features.

2. Not all phone brands are compatible with android studio

There is a feature in android studio, to test the application in real-time on a phone , to see exactly if it works or not on the spot, and any change would result in an immediate change on the device.

However, none of our phones were compatible with that feature, so instead we had to manually convert the application to a format that the phone can install, and then find a way to send it and install it. It cost us a lot of time and energy, since it means that every time a feature was added, we couldnt simply see the result, but had to follow this long process in order to be able to see the result of our changes.

VII. Glossary

Below is the explanation of some technical terms that were used in this report !

> **Software**
> Physical computer components (the screen, battery, microchips) are the physical body of a device.
>
> A software is a set of instructions that tells these physical components how to behave.
>
> I.e : **Apps**, **mobile games**, **web browsers**, and operating systems are all different types of software.

Figure 12: Example of softwares that you may know

Coding / Programming
Computers do not natively understand human languages like English or Tagalog; they only understand basic electrical signals (on or off). **Coding** is the act of writing step-by-step recipes or instructions in a specialized language (like Kotlin) that bridges the gap, translating human ideas into commands a computer can execute.

Figure 13: Your favorite app might be written in one of these programming languages

Bug
A fancy word for a mistake, error, or flaw in a software program that causes it to behave unexpectedly. This could range from a minor visual glitch (like text overlapping) to a severe crash (the app shutting down entirely). “Debugging” is the detective work developers do to hunt down and fix these errors.

Framework
Think of a framework as a prefabricated house kit. Instead of forcing a carpenter to cut down trees and forge custom nails just to build a kitchen, the kit provides pre-made walls and structures. In software, a framework (like Jetpack Compose) provides built-in components for common features like scroll lists, buttons, and text fields so developers don’t have to build them from absolute scratch.

In short, it’s a set of tools and pre-written code that helps developers build applications more efficiently, ensuring consistency and reducing the likelihood of errors.

Operating System (OS)
The master software that acts as the supervisor of a device. It manages the physical hardware resources and allocates them to the apps you run. It is the reason an app can easily request to use your phone’s screen or processor without needing to know the technical engineering details of that specific phone model.

IDE (Integrated Development Environment)
The ultimate digital workshop for a software developer. It is a specialized text editor (like Android Studio) that highlights typos in your code, auto-completes lines, warns you about potential bugs, and features a built-in virtual smartphone to test your game safely on your computer screen.

Native vs. Cross-Platform Applications
Native apps are coded exclusively for one type of device operating system using its official language (e.g., using Kotlin for Android). They run blindingly fast and fit perfectly on the phone, but won’t work anywhere else. **Cross-platform apps**

use a translation system to write code once and run it on both iPhones and Androids, which saves time but can slow down animation-heavy visual games.

Version Control (Git / GitHub)
A shared digital safety net for programmers. It tracks every single line of code edited by the team chronologically. If someone writes code that accidentally breaks the entire game, Version Control allows the team to instantly hit a massive "Undo" button and restore the app to a perfectly functional state from earlier in the day.

Low-Fidelity Wireframes & Mockups
A **wireframe** is a quick, rough design sketch (like drawing on a napkin or using an online sketching board) focused entirely on where elements sit, completely ignoring colors or fine art. A **mockup** is a polished static image (often generated to look real) showing stakeholders what the final product will visually look like before a single line of actual game code is written.

Bibliography

[1] I. Alemany-Arrebola, M. del M. Ortiz-Gómez, E. J. Lizarte-Simón, and Á. C. Mingorance-Estrada, "The attitudes towards mathematics: analysis in a multicultural context," *Humanities and Social Sciences Communications*, 2025, doi: 10.1057/s41599-025-04548-x.

[2] C. Tomasetto, K. Morsanyi, V. Guardabassi, and P. A. O'Connor, "Math anxiety interferes with learning novel mathematics contents in early elementary school," *Journal of Educational Psychology*, vol. 113, no. 4, pp. 753–769, 2021.

[3] E. A. Maloney, G. Ramirez, E. A. Gunderson, S. C. Levine, and S. L. Beilock, "Intergenerational effects of parents' math anxiety on children's math achievement and anxiety," *Psychological Science*, vol. 26, no. 9, pp. 1480–1488, 2015.

[4] A. Devine, F. Carey, J. Hill, and D. Szűcs, "Math anxiety in primary and secondary school students: Gender differences, standardized testing, and the influence of teachers," *Royal Society Open Science*, vol. 5, no. 1, p. 170566, 2018.

[5] J. M. Nagata, K. T. Ganson, C. jensen, and D. A. Christakis, "Adolescent Smartphone Use During School Hours," *JAMA Pediatrics*, vol. 179, no. 4, p. e245055, 2025, doi: 10.1001/jamapediatrics.2024.5055.

[6] J. Seaman and J. Seaman, "K-12 Curricula Discovery, Selection, and Adoption," technical report, 2023. [Online]. Available: https://www.bayviewanalytics.com/

[7] RAND Corporation, "Use of Instructional Materials in U.S. Schools: Findings from the American Educator Panels," technical report, 2022. [Online]. Available: https://www.rand.org/